Machine Learning–Driven Classification of Pulmonary Nodules from Volumetric CT Patches: A 3D Convolutional Network Study on LUNA16-Style Candidate Data

March 2025

---

1 Project Abstract

Pulmonary nodule detection in chest CT is a key step in lung cancer screening. We applied a 3D convolutional neural network (NoduleNet3D) to classify fixed-size volumetric patches (32^3 voxels) centred on candidate locations as nodule versus non-nodule, using a LUNA16-style pipeline with series-level train–validation–test splits. After loading candidate coordinates and CT volumes from `.mhd` sources, patches were extracted with a lung window (HU $[-1000, 400]$) and normalised to $[0, 1]$. Training was performed on a balanced subset of 1,359 candidates to mitigate class imbalance; validation and test sets retained the original distribution (test set: 42,625 samples, 100 nodules). We trained for 10 epochs with Adam (learning rate 10^{-3}) and CosineAnnealingLR, using BCEWithLogitsLoss. The model achieved a validation AUC of approximately 0.62 by epoch 10, with training loss decreasing from 0.64 to 0.49, indicating learned discriminative signal on the validation distribution. On the full test set, however, at a decision threshold of 0.5 the model predicted every sample as non-nodule, yielding test ROC-AUC 0.50 (no discrimination) and 0% sensitivity for nodules. A central finding was the **train–test distribution mismatch**: training on a balanced subset did not generalise to the heavily imbalanced test set under the default threshold, with likely contributions from limited training size, prior shift, and uncalibrated probabilities. These results support the use of balanced evaluation subsets, full training-set class weighting, and threshold or calibration analysis for deployment. Limitations include the single-dataset design, the small balanced training subset, and the need for external validation and optional test-set subsampling (e.g. `TEST_MAX_BATCHES`) to control memory when evaluating on the full candidate set.

Keywords: pulmonary nodule detection; 3D CNN; LUNA16; chest CT; volumetric patch classification; class imbalance; calibration; sensitivity.

---

2 Introduction and Background

Pulmonary nodules are focal opacities in the lung that may represent early malignancy. Their detection and characterisation in chest CT are central to lung cancer screening programmes. Computer-aided detection (CAD) systems that operate on volumetric CT data can assist radiologists by proposing candidate locations and classifying them as nodule or non-nodule. The

LUNA16 challenge and related benchmarks provide large sets of candidate coordinates with binary labels, derived from nodule annotations and negative samples, enabling supervised learning of patch-based classifiers.

Traditional CAD approaches often use hand-crafted features or 2D slice-based networks. Volumetric (3D) convolutional networks can exploit the full spatial context of a nodule and have been shown to improve discrimination when sufficient data are available. A common pipeline is to extract fixed-size 3D patches centred on each candidate, normalise intensity (e.g. with a lung window), and train a binary classifier. Performance is typically reported in terms of sensitivity, specificity, and area under the ROC curve (AUC); in screening contexts, sensitivity is often prioritised to minimise missed nodules.

The objectives of this study were: (1) to implement a 3D CNN pipeline for nodule-versus-non-nodule classification using LUNA16-style candidate lists and CT volumes; (2) to train on a balanced subset under a fixed computational budget and report training and validation metrics; (3) to evaluate on the full test set and on a balanced subset where applicable, and to interpret the discrepancy between validation and test performance; and (4) to document limitations and recommendations for threshold choice, calibration, and data scaling.

3 Statistical Analysis and Computational Environment

All data loading, model training, and evaluation were performed in Python using PyTorch (model definition and training loop), SimpleITK (reading `.mhd`/`.raw` volumes), pandas and NumPy (data handling), and scikit-learn (metrics: ROC-AUC, precision, recall, F1, confusion matrix). A fixed random seed was used for train–validation–test splits and for any stochastic sampling to ensure reproducibility. *Computational environment:* Training and inference were run on a workstation with CUDA-capable GPU; the same code runs on CPU with increased runtime. Optional limits (e.g. `TEST_MAX_BATCHES`) were available to bound memory use when evaluating on the full test set, as the dataset caches loaded CT volumes in RAM.

4 Computational Pipeline and Algorithmic Design

The analytical workflow was structured as a **multi-phase pipeline**: data loading and manifest alignment, patch extraction with coordinate mapping, balanced training subset selection, model training with validation monitoring, and test-set evaluation (full and, where implemented, balanced subset).

4.1 Data loading and patch extraction

Candidate tables (`candidates_train/val/test.csv`) and a series manifest (`series_manifest.csv`) were loaded. Each candidate row provides a `seriesuid`, world coordinates (`coordX`, `coordY`, `coordZ`), and binary label `class` (1 = nodule, 0 = non-nodule). For each candidate, the corresponding CT volume was loaded from `.mhd` (with co-located `.raw`); world coordinates were converted to voxel indices using the volume origin and spacing. A $32 \times 32 \times 32$ patch was cropped around the candidate centre (with boundary handling and padding when needed). Voxel intensities were clipped to the lung window (HU $[-1000, 400]$) and normalised to $[0, 1]$. Loaded volumes were cached in memory to avoid repeated disk reads; the cache was cleared before full test-set evaluation to reduce RAM use.

4.2 Model architecture and training

The classifier **NoduleNet3D** comprises three 3D convolutional blocks (Conv3d $\rightarrow$ BatchNorm3d $\rightarrow$ ReLU $\rightarrow$ MaxPool3d), reducing spatial dimensions to 4^3, followed by adaptive average pooling

to a single vector, then a two-layer MLP ($128 \to 64 \to 1$) with dropout 0.5. Output is a single logit; BCEWithLogitsLoss was used for training. Optimiser: Adam with learning rate 10^{-3}; scheduler: CosineAnnealingLR over 10 epochs. Training used the **balanced subset** of the training set (1,359 candidates from `candidates_train_balanced_subset.csv`) with shuffle; when the full training set is used, class-weighted sampling is applied. Validation was performed at the end of each epoch on the validation loader; validation AUC and loss were recorded.

4.3 Evaluation protocol

At test time, the trained model was applied to the test loader without shuffling. Predictions were collected as probabilities (sigmoid of logits); binary predictions were obtained with threshold 0.5. Confusion matrix, precision, recall, F1, and ROC-AUC were computed. Where a balanced test subset was evaluated, the same metrics were reported on that subset to allow interpretable sensitivity and specificity despite the imbalance of the full test set.

5 Data and Splits

The dataset was derived from LUNA16-style exports: a series manifest listing CT series (by `seriesuid`) and paths to `.mhd` files, and candidate tables with one row per candidate (series, coordinates, label). Splits were by series so that no series appeared in more than one of train, validation, or test. The **training** set was subsampled to a balanced subset of 1,359 candidates for the reported run. The **test** set comprised 42,625 candidates (42,525 non-nodules, 100 nodules), i.e. a prevalence of approximately 0.23% for the positive class. This severe imbalance, combined with a default threshold of 0.5, led to the model predicting all test samples as negative in the reported experiment.

6 Training Results

Table 1 reports training loss, validation loss, and validation AUC for each of the 10 epochs. Training loss decreased monotonically from 0.6356 to 0.4886; validation loss fluctuated (final 0.4749) and validation AUC improved from approximately 0.53 to approximately 0.62 by the end of training. Peak validation AUC in the run was 0.6267 (epoch 9). These results indicate that the model learned discriminative structure on the validation distribution and that overfitting was not severe within the 10-epoch budget.

Table 1: Training and validation metrics by epoch. Val AUC is the area under the ROC curve on the validation set.

Epoch	Train loss	Val loss	Val AUC
1	0.6356	0.4458	0.5347
2	0.5990	0.4634	0.5897
3	0.5949	0.4146	0.5867
4	0.5773	0.3730	0.5855
5	0.5631	0.4364	0.5901
6	0.5546	0.3536	0.5526
7	0.5286	0.4036	0.6120
8	0.5179	0.4318	0.6071
9	0.4887	0.4815	**0.6267**
10	0.4886	0.4749	**0.6213**

7 Test Set Evaluation (Full Set)

The trained model was evaluated on the full test set (42,625 samples) with decision threshold 0.5. Table 2 gives the confusion matrix; Table 3 summarises precision, recall, and F1 for each class. The model predicted **every** test sample as non-nodule: 42,525 true negatives, 0 true positives, 0 false positives, and 100 false negatives. Thus **test ROC-AUC** was **0.5000** (no discrimination), **sensitivity (recall for nodule)** was **0.00**, and accuracy was 1.00 only because of the extreme class imbalance.

Table 2: Confusion matrix on the full test set (threshold 0.5).

	Predicted non-nodule	Predicted nodule
True non-nodule	42,525	0
True nodule	100	0

Table 3: Classification metrics on the full test set (threshold 0.5).

Metric	Non-nodule	Nodule
Precision	1.00	— (no positive predictions)
Recall	1.00	**0.00**
F1-score	1.00	0.00

8 Interpretation and Train–Test Mismatch

Validation AUC (∼0.62) indicates that the model has learned to rank nodule versus non-nodule reasonably well on the validation distribution, which is consistent with the use of a balanced training subset and a validation set that may share similar balance or sampling. The full test set, by contrast, has a nodule prevalence of about 0.23%. Under a default threshold of 0.5, the model never predicted the positive class; hence test AUC was 0.50 and sensitivity was 0.

Possible causes include: (1) **prior shift**—the test set has a much lower prevalence than the balanced training subset, so the learned decision boundary (in logit or probability space) may not translate to the test prior without recalibration or threshold adjustment; (2) **limited training data**—1,359 samples may be insufficient for the model to generalise to the full candidate distribution; (3) **uncalibrated probabilities**—raw sigmoid outputs may be systematically low for nodules when the model is uncertain, so that no prediction exceeds 0.5. The pipeline (data load, 3D CNN, training loop, evaluation) runs correctly; the discrepancy is therefore attributable to evaluation setup and train–test distribution rather than to implementation error.

9 Discussion

9.1 Summary of findings

This study implemented a 3D CNN (NoduleNet3D) for binary classification of pulmonary nodule candidates from volumetric CT patches in a LUNA16-style setting. Training on a balanced subset of 1,359 candidates over 10 epochs yielded a validation AUC of approximately 0.62 and decreasing training loss, indicating learned discriminative ability on the validation set. On the full test set (42,625 samples, 100 nodules), the model predicted all samples as non-nodule at threshold 0.5, giving test AUC 0.50 and 0% sensitivity. The central finding is the **train–test distribution mismatch**: performance on a balanced validation set does not carry over to the heavily imbalanced test set under the default threshold. The results support reporting metrics

on a balanced test subset, using class-weighted training on the full training set, and performing threshold or calibration analysis before deployment.

9.2 Limitations

The analysis is based on a single dataset and a single train–validation–test split; **external validation** was not performed. The **training subset** was limited to 1,359 balanced candidates; the full training set was not used in the reported run. **No calibration** (e.g. isotonic or Platt) was applied to the model outputs, so predicted probabilities may not reflect empirical event rates on the test distribution. The **default threshold** 0.5 was used throughout; no sweep over operating points was reported. The test set is highly imbalanced, so accuracy is not informative; sensitivity and AUC (or AUC-PR) on a balanced subset would be more appropriate for comparison. Memory constraints may require limiting the number of test batches (`TEST_MAX_BATCHES`) when evaluating on the full set, which was not applied in the run that produced the reported full-set metrics.

9.3 Comparison with baseline and recommendations

No explicit baseline (e.g. random or constant predictor) was reported; the validation AUC ($\sim$0.62) versus test AUC (0.50) illustrates that in-distribution validation can be optimistic when the test distribution differs. Recommendations for improvement include: (1) evaluate on a **balanced test subset** (e.g. 2:1 neg:pos) to obtain interpretable sensitivity, specificity, and AUC; (2) train on the **full training set** with class-weighted sampling to improve generalisation; (3) perform **threshold sweep** or **calibration** (e.g. isotonic regression on a validation subset) to choose an operating point or to improve probability interpretability; (4) consider **precision–recall AUC** and sensitivity at several thresholds; (5) add **3D data augmentation** and, where possible, more training data to improve robustness.

10 Future Work and Conclusion

Future work may extend the pipeline in several directions. (1) **Full training set** with class-weighted or focal loss could be evaluated to improve calibration and test-set sensitivity. (2) **Post-hoc calibration** (isotonic or Platt) could be applied so that predicted probabilities reflect observed nodule prevalence on a validation or balanced subset. (3) **Multi-threshold reporting** (e.g. sensitivity at 0.2, 0.3, 0.5) and **AUC-PR** would better characterise performance on imbalanced test sets. (4) **External validation** on another cohort or another centre would strengthen generalisability claims. (5) Integration with a **candidate generation** stage (e.g. detector or rule-based proposal) would yield an end-to-end CAD pipeline.

Conclusion. This study demonstrated a 3D CNN pipeline for pulmonary nodule versus non-nodule classification on LUNA16-style data. Validation AUC reached approximately 0.62, but on the full imbalanced test set the model predicted all samples as non-nodule at threshold 0.5, yielding test AUC 0.50 and 0% sensitivity. The train–test distribution mismatch underscores the importance of balanced evaluation, class-weighted training, and threshold or calibration analysis before deployment. With these extensions and external validation, the pipeline could contribute to reproducible benchmarks and to the design of CAD systems for lung nodule detection.

11 Data and Code Availability

The analysis used a LUNA16-style dataset comprising a series manifest (`series_manifest.csv`) and candidate tables (`candidates_train.csv`, `candidates_val.csv`, `candidates_test.csv`, and `candidates_train_balanced_subset.csv`) under `dataset_export/`. CT volumes were

stored as `.mhd`/`.raw` in subset directories. The computational pipeline (patch extraction, NoduleNet3D, training and evaluation) was implemented in a Jupyter notebook (`nodule_detection.ipynb`). Configuration: `PATCH_SIZE=32`, `BATCH_SIZE=32`, `EPOCHS=10`, `LR=1e-3`, with a fixed random seed for reproducibility. Code and documentation are maintained in the project repository.

References

1. Armato III, S. G., et al. (2011). The Lung Image Database Consortium (LIDC) and Image Database Resource Initiative (IDRI): A completed reference database of lung nodules on CT scans. *Medical Physics*, 38(2), 915–931.

2. Setio, A. A. A., et al. (2016). Validation, comparison, and combination of algorithms for automatic detection of pulmonary nodules in computed tomography images: The LUNA16 challenge. *Medical Image Analysis*, 42, 1–13.

3. Paszke, A., et al. (2019). PyTorch: An imperative style, high-performance deep learning library. *Advances in Neural Information Processing Systems*, 32, 8026–8037.

4. Pedregosa, F., et al. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830.